

EXPERIMENT 9: FORWARD CHAINING

AIM

To implement and demonstrate the concept of forward chaining in PROLOG using facts, rules, and dynamic knowledge representation.

OBSERVATION

1. Simple Inference

Problem: Infer a new fact when the required initial facts are known.

Program:

```
:- dynamic known/1.

% Initial knowledge
start :-
    assert(known(a)),
    assert(known(b)),
    infer.

% Rule
infer :-
    known(a),
    known(b),
    \+ known(c),
    assert(known(c)),
    write('Inferred c'),
    nl.

% Query
% ?- start.
```

Output:

Inferred c

Observation:

The initial facts known(a) and known(b) are added to the knowledge base. The rule checks whether both facts are known and c is not already known. Since the conditions are satisfied, known(c) is added and the message "**Inferred c**" is displayed.

2. Fire Alert System

Problem: Infer a fire alert when smoke and heat are detected.

Program:

```
:- dynamic known/1.

% Rule to infer fire
check :-
    known(smoke),
    known(heat),
    assert(known(fire)),
    write('Fire Alert!'),
    nl.

% Start rule
start :-
    assert(known(smoke)),
    assert(known(heat)),
    check.

% Query
% ?- start.
```

Output:

Fire Alert!

Observation:

The facts `known(smoke)` and `known(heat)` are added to the knowledge base. The rule checks for both conditions. When both are present, `known(fire)` is inferred and the message **"Fire Alert!"** is displayed.

3. Water Purity Detection

Problem: Infer water pollution when cloudy water and bad smell are detected.

Program:

```
:- dynamic known/1.
```

```
% Rule to infer pollution
```

```
infer_pollution :-  
    known(cloudy_water),  
    known(bad_smell),  
    assert(known(polluted)),  
    write('Water is polluted'),  
    nl.
```

```
% Start rule
```

```
start
```